

# **"Verified Abbreviated Zeta 256 bits (VAZ256): Esquema de Firma Digital Post-Cuántica con Claves de 32 bytes, basado en Dilithium5 y SHAKE-256"**

**Autor:** Fran Luis Vazquez Alonso

**Fecha:** Junio 2026

**Versión:** 0.2.0

**Repositorio:** <https://www.github.com/GrayLupus/vaz256>

**Licencia:** GPL-3.0

## **Resumen**

---

VAZ256™ es un esquema de firma digital post-cuántico que combina Dilithium5 y SHAKE256 para lograr firmas resistentes a ataques cuánticos utilizando claves públicas y claves privadas ultracompactas de 32 bytes sin pérdida de nivel de seguridad. El objetivo es ofrecer una alternativa eficiente en generación de claves, firmas y verificación, con seguridad equivalente a NIST Level 5, ideal para sistemas donde el almacenamiento de claves es crítico. Mediante la compresión mediante funciones hash de claves públicas Dilithium5 y esquemas de firma que combinan firmas y claves públicas Dilithium, logrando así reducir en más de un 98% el tamaño de las claves frente a Dilithium5, y aún así manteniendo firmas digitales más pequeñas que esquemas ya más establecidos como SPHINCS+. Los resultados muestran tiempos eficientes de generación de claves y verificación de firmas, superando incluso a Falcon1024 en eficiencia.

**Palabras clave:** criptografía, post-cuántica, firma digital, dilithium, hash

## **Introducción**

---

La criptografía post-cuántica enfrenta el desafío de equilibrar seguridad, tamaño de claves y eficiencia. Esquemas como Falcon (claves pequeñas pero complejo en implementación) o Dilithium (eficiente pero con claves públicas muy grandes) o SPHINCS+ (poco eficiente en cuanto a tamaño de firmas), limitan su adopción en sistemas con restricciones de almacenamiento, como blockchain. VAZ256 surge como una solución híbrida que comprime claves públicas mediante SHAKE256 y utiliza firmas

con lógica Dilithium5, manteniendo la seguridad NIST Level 5. Su diseño permite claves memorizables (32 bytes) y verificación rápida, ideal para sistemas que prioricen la seguridad, la eficiencia y no sea tan influyente el espacio de almacenamiento en firmas digitales, o en redes con mecanismos de optimización como el pruning de firmas.

VAZ256 aborda este desafío de reducir el tamaño de las claves públicas mediante un enfoque innovador que combina la compresión de claves con firmas de doble componente, logrando un equilibrio óptimo entre seguridad y rendimiento. Los objetivos del trabajo incluyen explicar la lógica y fundamentos de este nuevo esquema y demostrar su viabilidad a través de la comparación de su rendimiento con otros esquemas post-cuánticos ya establecidos.

## Fundamentos Teóricos

---

La idea base para el desarrollo de VAZ256 fue implementarlo en Rust puro y sin dependencias externas. El código y la lógica de VAZ256 fue redactado realizando modificaciones sobre el código de referencia de Dilithium5 en CRYSTALS-Dilithium, de Quantum-Blockchains, disponible en Github.

Al ser VAZ256 una modificación sobre Dilithium5 es necesario empezar haciendo una breve explicación de la lógica y funcionamiento de Dilithium5 y una vez explicado en brevedad, pasar a explicar a VAZ256 en mayor profundidad.

\*Para una mayor profundidad sobre el uso e implementación de Dilithium y las exigencias de su estándar, puede dirigirse a <https://pq-crystals.org/dilithium/data/dilithium-specification-round3-20210208.pdf> para abordar en mayor profundidad las características de Dilithium5.

## 1. Fundamentos Teóricos y Funcionamiento de Dilithium5

---

### 1.1. Estructura Matemática

Dilithium5 es un esquema de firma digital basado en retículos (lattices) que utiliza dos problemas computacionales fundamentales:

1. **Module Learning With Errors (MLWE):** Dada una matriz  $A \in \mathbb{R}_q^{k \times l}$  y  $t = As_1 + s_2$ , es difícil recuperar  $s_1, s_2$ .

2. **Module Short Integer Solution (MSIS):** Dada  $A$ , es difícil encontrar  $z$  pequeño tal que  $Az = 0$ .

Estos problemas son resistentes a ataques cuánticos y definen la seguridad de Dilithium5 (nivel 5 del NIST, equivalente a 256 bits de seguridad).

## 1.2. Parámetros Clave

Dilithium5 opera sobre un anillo polinomial  $R_q = \mathbb{Z}_q[x]/(x^n + 1)$ , donde:

- $q = 8.380.417$  (primo de 23 bits, definido como  $2^{23} - 2^{13} + 1$ )
- $n = 256$  (grado fijo del polinomio)
- $(k, l) = (8, 7)$
- $\eta = 2$
- $\gamma_1 = 2^{19}$
- $\tau = 60$
- $\omega = 75$

### Tamaños:

- Clave pública (pk): 2592 bytes (matriz  $A \in R_q^{k \times l}$  y vector  $t$  comprimido).
- Clave privada (sk): 4864 bytes (vectores  $s_1, s_2$ , semilla para  $A$ , y pk).
- Firma ( $\sigma$ ): 4595 bytes (vector de compromiso  $z$ , desafío  $c$ , y vector  $h$ ).

## 1.3. Generación de Claves

1. **Semilla y matriz pública:** Se elige una semilla uniforme  $\rho \leftarrow \{0,1\}^{256}$ .  
Se deriva la matriz  $A \in R_q^{k \times l}$  mediante  $\text{ExpandA}(\rho)$  usando SHAKE-128.
2. **Vectores secretos:**  $s_1 \leftarrow S_{\eta}^l$  (vector con coeficientes en  $[-2, 2]$ ,  $\eta = 2$ ).  
 $s_2 \leftarrow S_{\eta}^k$  (vector pequeño de ruido con misma distribución).
3. **Cálculo de clave pública:**  $t = As_1 + s_2 \in R_q^k$ . Descomposición:  $t = t_1 \cdot 2^{13} + t_0$  (usando  $\text{power2round}$ ). Clave pública  $\text{pk} = (\rho, t_1)$ . Clave privada  $\text{sk} = (\rho, s_1, s_2, t_0)$ .

## 1.4. Proceso de Firma

1. **Hash del mensaje:**  $\mu = \text{CRH}(\text{CRH}(\text{pk}) || M)$  usando SHAKE-256.
2. **Generación de máscara:**  $y \leftarrow S_{\gamma_1-1}^l$  donde  $\gamma_1 = 2^{19}$ .
3. **Compromiso:**  $w = Ay \in R_q^k$ .  $w_1 = \text{HighBits}(w)$ .

4. **Desafío:**  $c = H(\mu || w_1) \in B_T$  (polinomio con exactamente 60 coeficientes  $\pm 1$ ).
5. **Respuesta:**  $z = y + cs_1$ . Rechazar si  $\|z\|_\infty \geq \gamma_1 - \beta$  ( $\beta = 120$ ).
6. **Pistas:**  $h = \text{MakeHint}(-ct_0, w - cs_2)$ . Rechazar si número de 1's en  $h > \omega$  ( $\omega = 75$ ).
7. **Firma:**  $\sigma = (z, c, h)$ .

### 1.5. Verificación de la Firma

1. **Reconstrucción:**  $w' = Az - ct_1 \cdot 2^{13}$ .
2. **Corrección:**  $w_1' = \text{UseHint}(h, w')$ .
3. **Verificaciones:**  $c' = H(\mu || w_1')$ . Aceptar si:  $c' == c$ ,  $\|z\|_\infty < \gamma_1 - \beta$ , número de 1's en  $h \leq \omega$ .

## 2. Fundamentos Teóricos y Funcionamiento de VAZ256

---

### 2.1. Fundamentos Matemáticos

VAZ256 es un esquema híbrido que integra dos componentes críticos:

1. **Dilithium5:** Esquema de firma post-cuántico basado en retículos, cuya seguridad como antes se mencionaba se fundamenta en los problemas MLWE (Module Learning With Errors) y MSIS (Module Short Integer Solution).
2. **SHAKE-256:** Función hash extensible de la familia SHA-3, resistente a colisiones y preimagen.

### 2.2. Parámetros Clave

- **Clave privada VAZ256:** 32 bytes generados de manera aleatoria para mayor seguridad (256 bits).
- **Clave pública VAZ256:** 32 bytes (hash SHAKE256 de 256 bits de longitud sobre los 2592 bytes de la clave pública de Dilithium5).
- **Firma VAZ256:** 7187 bytes (4595 bytes para la firma digital Dilithium5 + 2592 bytes para la clave pública Dilithium5).
- **Seguridad:** Post-cuántica: reducida a la dificultad de resolver MLWE y MSIS en parámetros Dilithium5. Clásica: resistencia a colisiones y preimagen de SHAKE-256 ( $2^{128}$  operaciones para ataques).

## 2.3. Generación de Claves VAZ256

1. **Generación de Clave Privada VAZ256:** Se genera una clave privada (secret) de 256 bits de manera aleatoria.
2. **Generación de Clave Pública VAZ256:** La clave privada de VAZ256 (secret) se usa como semilla para generar determinísticamente un par de claves Dilithium5, utilizando la función `Dilithium5Keypair` que aplica funciones hash SHAKE256 con diferentes parámetros para obtener el par de clave privada Dilithium5 (`dilithium5_sk`) y a la vez utilizando la metodología establecida para Dilithium5, se calcula la clave pública Dilithium5 (`dilithium5_pk`). La clave pública Dilithium5 de 2592 bytes se comprime a un estándar de 32 bytes mediante la función hash SHAKE256, y el hash de 32 bytes resultante constituye la clave pública VAZ256 (`vaz256_pk`).

## 2.4. Proceso de Firma VAZ256

1. **Regeneración del par Dilithium5:** A partir de la clave privada VAZ256 (secret), y usando la función `Dilithium5Keypair` se regenera el par de claves privadas y públicas de Dilithium5 (`dilithium5_sk`; `dilithium5_pk`).
2. **Cálculo de firma Dilithium5 del mensaje:** Dado un mensaje M, se calcula el hash de dicho mensaje y se aplica exactamente la misma metodología de cálculo de la firma que se usa para firmar un mensaje M en Dilithium5 y se obtiene como resultado una firma Dilithium5 (`dilithium5_signature`) de 4595 bytes específica para dicho mensaje M.
3. **Emisión de la firma VAZ256:** La firma final VAZ256 (signature) es la tupla entre la firma Dilithium5 (`dilithium5_signature`) y la clave pública Dilithium5 (`dilithium5_pk`):  
$$\text{signature} = (\text{dilithium5\_signature}, \text{dilithium5\_pk})$$

## 2.5. Verificación de la Firma VAZ256

1. **Validación de la clave pública:** El primer paso para la validación consiste en la separación de la firma VAZ256 (signature) en sus dos componentes claves, la firma Dilithium5 (`dilithium5_signature`) de 4595 bytes y los 2592 bytes finales de la clave pública Dilithium5 (`dilithium5_pk`). Entonces se calcula el hash SHAKE256 a 32 bytes de la clave pública Dilithium5 y si coinciden exactamente se acepta este paso en la firma VAZ256 y se pasa a la siguiente fase de comprobación de la firma.

2. **Verificación de la firma Dilithium5:** Con los dos componentes separados de la firma VAZ256 se confirma si la firma Dilithium5 (dilithium5\_signature) es válida para la clave pública Dilithium5 (dilithium5\_pk) y para dicho mensaje M utilizando la misma metodología de verificación de firmas digitales de Dilithium5.

## 2.6. Ataques Mitigados

1. **Sustitución de clave pública:** Imposible sin romper la resistencia a preimagen de SHAKE-256.
2. **Falsificación de firmas:** Requeriría resolver MLWE/MSIS (Dilithium5) o encontrar colisiones en SHAKE-256, pero dichas colisiones en SHAKE-256 deben resultar en otra clave pública Dilithium5 de 2592 bytes de la cual se deberían de conocer sus claves privadas para poder emitir firmas Dilithium5 válidas, por lo tanto la seguridad del esquema vuelve a recaer en resolver MLWE/MSIS (Dilithium5), ratificando que VAZ256 y Dilithium5 tienen el mismo nivel de seguridad y representan el mismo desafío.
3. **Fugas de memoria:** La clave privada VAZ256 y que constituye la semilla generadora de las claves Dilithium5 se borra de manera segura gracias a técnicas de zeroization.

## 3. Rendimiento

---

### 3.1. Análisis Comparativo Detallado (RockChip RK3562 2.0 GHz)

**Tabla 1: Comparativa de esquemas post-cuánticos (Nivel 256 bits)**

| Esquema    | Clave Privada | Clave Pública | Tamaño Firma | KeyGen   | Sign     | Verify  |
|------------|---------------|---------------|--------------|----------|----------|---------|
| VAZ256     | 32 bytes      | 32 bytes      | 7187 bytes   | 1.33 ms  | 5.00 ms  | 1.01 ms |
| Dilithium5 | 4864 bytes    | 2592 bytes    | 4595 bytes   | 1.30 ms  | 2.10 ms  | 0.90 ms |
| Falcon1024 | 1281 bytes    | 1793 bytes    | 1330 bytes   | 12.50 ms | 17.20 ms | 2.30 ms |
| SPHINCS+   | 64 bytes      | 32 bytes      | 17220 bytes  | 0.15 ms  | 15.50 ms | 0.30 ms |

### 3.2. Ventajas de VAZ256

- **Claves Ultracompactas:** 32 B vs. 2592 B en Dilithium5, ideal para sistemas con restricciones de almacenamiento (ej: chips IoT).
- **Eficiencia en Verificación:** 1.01 ms vs. 2.30 ms de Falcon1024, crítico en blockchain con alta frecuencia de transacciones.
- **Resistencia Cuántica Total:** Combina seguridad en retículos (Dilithium5) y funciones hash (SHAKE256), teóricamente resistentes a ataques cuánticos conocidos.

### 3.3. Compromisos y Limitaciones

- **Firmas Grandes:** 7187 bytes pueden ser prohibitivos para redes con ancho de banda limitado o sistemas donde el almacenamiento sea un recurso crítico y por algún motivo no puede ser posible implementar otras estrategias de seguridad que no sean almacenar un histórico de todas las firmas válidas emitidas.
- **Dependencia de SHAKE256:** Un hipotético avance en colisiones SHAKE256 debilitaría la compresión de claves aunque la seguridad de Dilithium5 permanecería intacta y para poder emitir una firma válida todavía sería necesario conocer las claves privadas que pudieran generar esa clave pública Dilithium5 que resultó en la colisión, pero la mayor vulnerabilidad podría ocurrir en el proceso de cálculo del hash del mensaje M y la apertura a la remota posibilidad de falsificar mensajes distintos bajo un mismo hash para entonces reutilizar firmas válidas ya anteriormente emitidas.

### 3.4. Optimizaciones Clave

- Operaciones en tiempo constante para operaciones sensibles (ej: keygen, sign, comparación de hashes).
- Prevención ante Ataques de Canal Lateral: la implementación en Rust evita fugas mediante acceso restringido a registros de hardware y el uso de zeroization para la limpieza segura de memoria.

## Conclusiones

---

1. VAZ256 logra claves públicas/privadas de 32 bytes sin comprometer seguridad post-cuántica, superando a Falcon y SPHINCS+ en eficiencia de KeyGen y logra redefinir el Balance Clave-Firma: priorizando claves

minúsculas (32 B) sin aumentar excesivamente el tamaño de las firmas (7 KB vs. 17 KB de SPHINCS+).

2. El esquema dual Dilithium5-SHAKE256 permite verificación ultra-rápida (1.01 ms), ideal para blockchain de alta frecuencia y IoT. También la combinación de retículos y funciones hash proporciona redundancia ante futuros avances criptoanalíticos.
3. Las firmas grandes (7187 B) son viables en sistemas con pruning, donde el almacenamiento a largo plazo recae en claves compactas, o en sistemas que monitorean el histórico de firmas mediante la aplicación eficiente de árboles de Merkle para evitar el almacenamiento total histórico.

En resumen VAZ256 es una extensión "práctica y segura" de Dilithium5 que optimiza el almacenamiento de claves públicas sin comprometer la seguridad. Su seguridad se reduce a la robustez de Dilithium5 y SHAKE-256, ambos estandarizados y ampliamente analizados. Al integrar firmas autosuficientes y compresión mediante hashing, ofrece un equilibrio único entre eficiencia y resistencia post-cuántica.

## Referencias

---

- Bai, S., Ducas, L., Kiltz, E., Lepoint, T., Lyubashevsky, V., Schwabe, P., Seiler, G., & Stehlé, D. (2021). CRYSTALS-Dilithium: Algorithm Specifications and Supporting Documentation (NIST Round 3 Submission). National Institute of Standards and Technology. <https://pq-crystals.org/dilithium/data/dilithium-specification-round3-20210208.pdf>
- Ducas, L., Kiltz, E., Lepoint, T., Lyubashevsky, V., Schwabe, P., Seiler, G., & Stehlé, D. (2018). FALCON: Fast-Fourier lattice-based compact signatures over NTRU. National Institute of Standards and Technology. <https://falcon-sign.info/falcon.pdf>
- National Institute of Standards and Technology. (2023). Post-Quantum Cryptography Standardization Project. <https://csrc.nist.gov/projects/post-quantum-cryptography>